

Embedded Wireless Design Lab 1

Name: Marc Pham

```
In [95]: import numpy as np
import matplotlib.pyplot as plt
from IPython.display import display_pdf
```

Experiment 1: Basic Flow and File Transfer

Question 1

Number of Samples N = Samples per second * record samples in seconds = $10 \text{ MS} / \text{s} * 1 \text{ s} = 10 * 10^6 \text{ Samples} = 10^7 \text{ Samples}$

One float32 sample is 4 bytes, and one complex64 sample is 8 bytes. This is because a float32 sample stores a single 4-byte floating point number, while one complex64 sample stores one 4-byte in-phase (I) component and one 4-byte quadrature (Q) component. With $N = 10,000,000$ samples, the float32 file is 40,000,000 bytes = 40MB, and the complex64 file is 80,000,000 bytes = 80MB.

```
In [96]: f0 = 10
Fs = 10 * 10 ** 6 # Sampling rate in samples per second
T = 1 # Record duration in seconds.
N = int(Fs * T) # Number of Samples
n = np.arange(N) # Sample indices
t = n / Fs # Sample times

print(f"Number of Samples N = {N}")
```

Number of Samples $N = 10000000$

```
In [97]: x = np.sin(2 * np.pi * f0 * t)
```

```
In [4]: x.astype(np.float32).tofile("signal_f32.dat")
x.astype(np.complex64).tofile("signal_c64.dat")
```

Verification: File sizes match what is listed with `ls -l`. `signal_f32.dat` is 40,000,000 bytes and `signal_c64.dat` is 80,000,000 bytes.

```
In [98]: !ls -l signal_f32.dat
!ls -l signal_c64.dat

-rw-rw-r-- 1 mephm mephm 40000000 Aug 29 00:47 signal_f32.dat
-rw-rw-r-- 1 mephm mephm 80000000 Aug 29 00:47 signal_c64.dat
```

Question 2

The final path used by GNU Radio is `/home/mephm/ECE7377/signal_f32.dat`

Question 3

`samp_rate = 10,000,000` samples per second because the sample rate `$F_s$` is set to 10 MegaSamples per second, which is $10 * 10^6$ Samples per second.

The expected output file size is 40MB. Changing the Throttle rate does not change the file size or sample values because the Throttle controls the data processing rate, not the data itself.

```
In [54]: !python3 Lab1/lab1_exercisel.py
```

```
In [55]: x_5M = np.fromfile("Lab1/signal_f32_Throttle5M.dat", dtype=np.float32)
x_10M = np.fromfile("Lab1/signal_f32_Throttle10M.dat", dtype=np.float32)
```

Verification: File size doesn't change

```
In [64]: !cd Lab1 && ls -l | grep -E 'signal_f32_Throttle*'
```

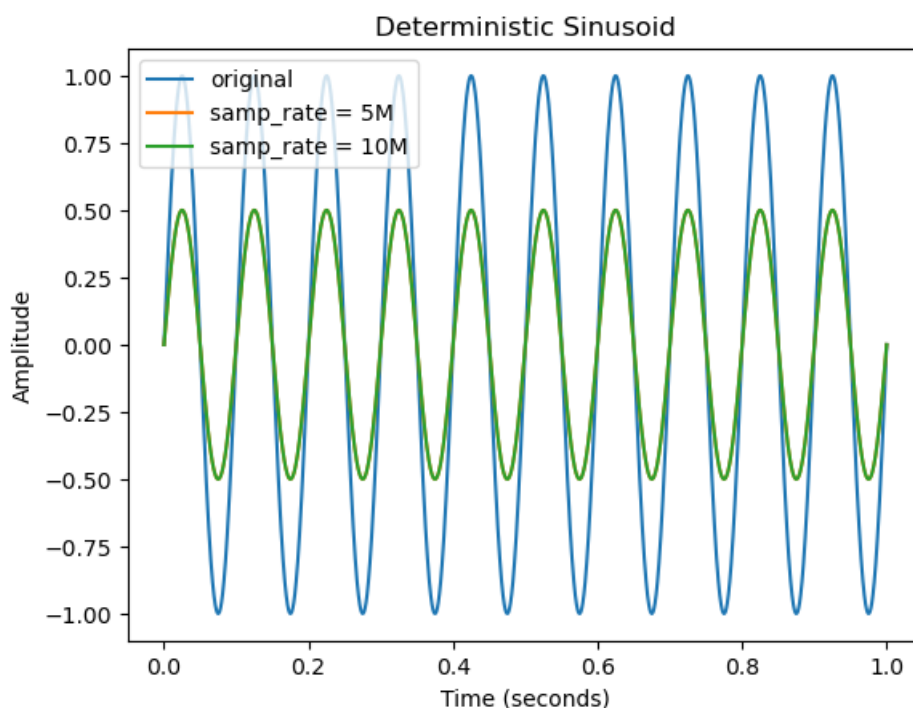
```
-rw-rw-r-- 1 mephah mephah 40000000 Aug 29 03:19 signal_f32_Throttle10M.dat
-rw-rw-r-- 1 mephah mephah 40000000 Aug 29 03:18 signal_f32_Throttle5M.dat
```

Verification: Sample values don't change

```
In [48]: all(x_changed == x_output)
```

Out[48]: True

```
In [65]: plt.figure()
plt.plot(t, x, linewidth=1.5, label='original')
plt.plot(t, x_5M, linewidth=1.5, label='samp_rate = 5M')
plt.plot(t, x_10M, linewidth=1.5, label='samp_rate = 10M')
plt.xlabel("Time (seconds)")
plt.ylabel("Amplitude")
plt.legend(loc='upper left')
plt.title("Deterministic Sinusoid")
plt.show()
```



Question 4

$f_0 = 10$ Hz

Period $T_0 = \frac{1}{f_0} = 0.1$ seconds, so a one-half period delay $\tau = \frac{T_0}{2} = 0.05$ seconds.

Delay $D = \tau f_s = 0.05 \text{ sec} * 10^7 \text{ Samples per sec} = 500,000$ samples

Phase $\phi = -360^\circ f_0 \tau = -360^\circ * 10 \text{ Hz} * 0.05 \text{ sec} = -180^\circ$

With a 500,000 sample delay, there will be $500k \text{ samples} * 1 \text{ float32 per sample} * 4 \text{ bytes per float32} = 2\text{MB}$ increase to the delayed file size on top of the original file size. Thus, the file sizes are as follows...

Original File Size: 40MB, Multiplied File Size: 40MB, and Delayed File Size: 42MB

```
In [66]: !python3 Lab1/lab1_exercise1.py
```

```
In [67]: x_mult = np.fromfile("Lab1/signal_f32_multiplied.dat", dtype=np.float32)
x_delay = np.fromfile("Lab1/signal_f32_delayed.dat", dtype=np.float32)
```

```
In [69]: !cd Lab1 && ls -l | grep -E 'signal_f32_mult*'
!cd Lab1 && ls -l | grep -E 'signal_f32_delay*'
```

```
-rw-rw-r-- 1 mephah mephah 40000000 Aug 29 03:42 signal_f32_multiplied.dat
-rw-rw-r-- 1 mephah mephah 42000000 Aug 29 03:42 signal_f32_delayed.dat
```

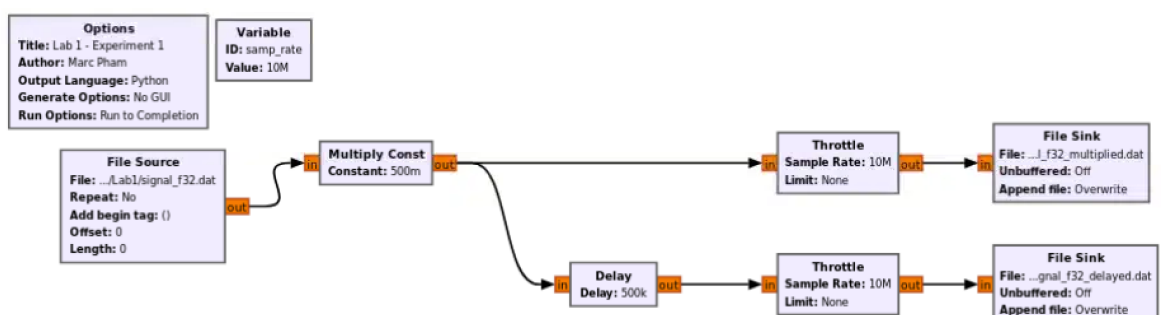
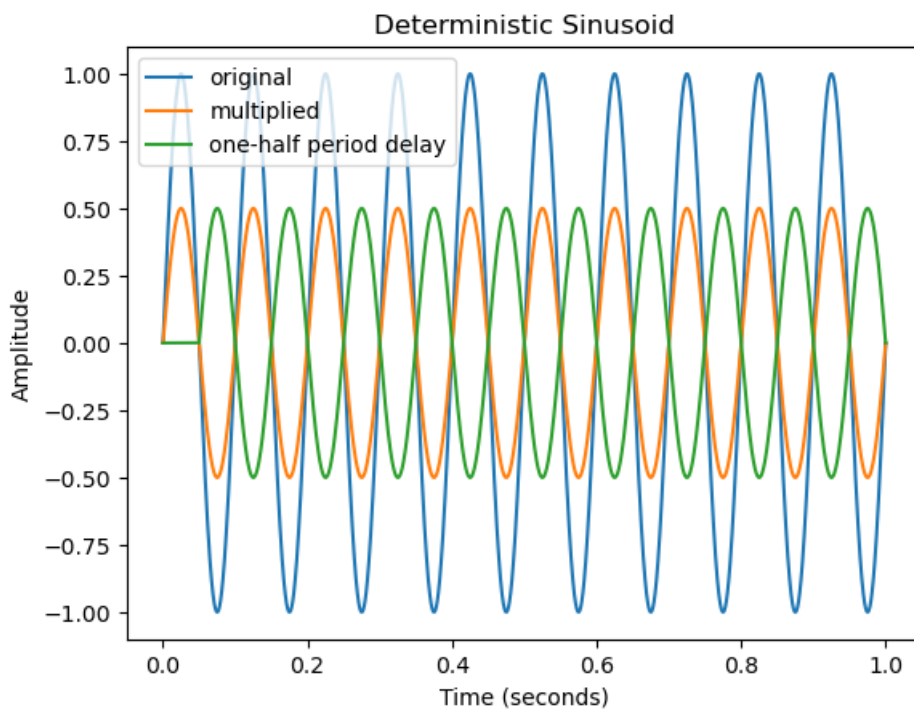
Question 5

lab1_exercise1.grc file saved

Question 6

The observed amplitude change is that the amplitude of the output signal is 0.5 of the original signal because we the Multiply Const block is set to 0.5 and multiplies all samples of the input signal by 0.5. The Delay block shift the signal by one-half period (0.05 seconds), which corresponds to a 180° phase shift. This causes the sinusoid to have a half-period phase reversal.

```
In [71]: plt.figure()
plt.plot(t, x, linewidth=1.5, label='original')
plt.plot(t, x_mult, linewidth=1.5, label='multiplied')
plt.plot(t, x_delay[:len(x_mult)], linewidth=1.5, label='one-half period delay')
plt.xlabel("Time (seconds)")
plt.ylabel("Amplitude")
plt.legend(loc='upper left')
plt.title("Deterministic Sinusoid")
plt.show()
```

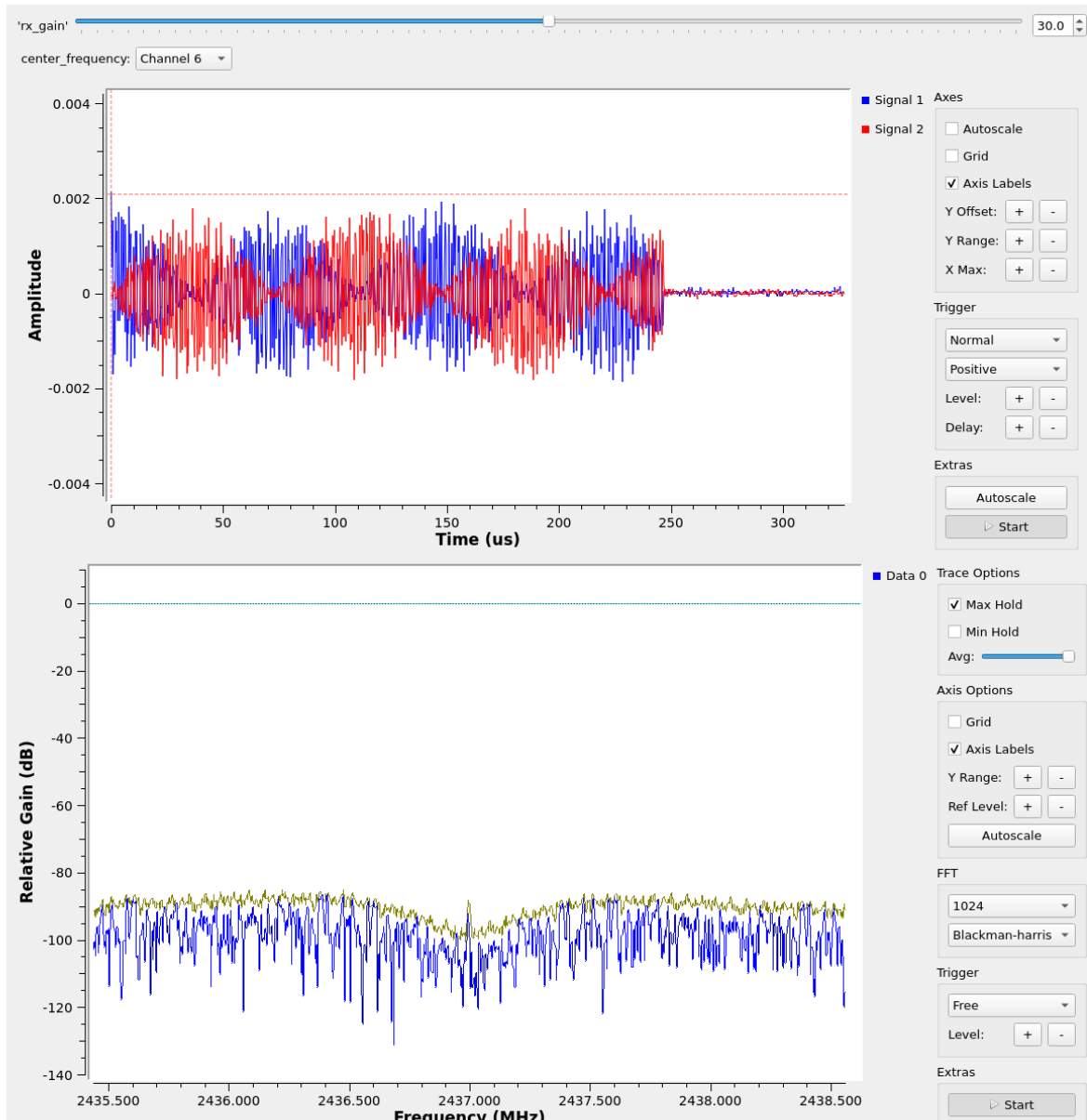


Experiment 2: Using an N310 as a Receiver

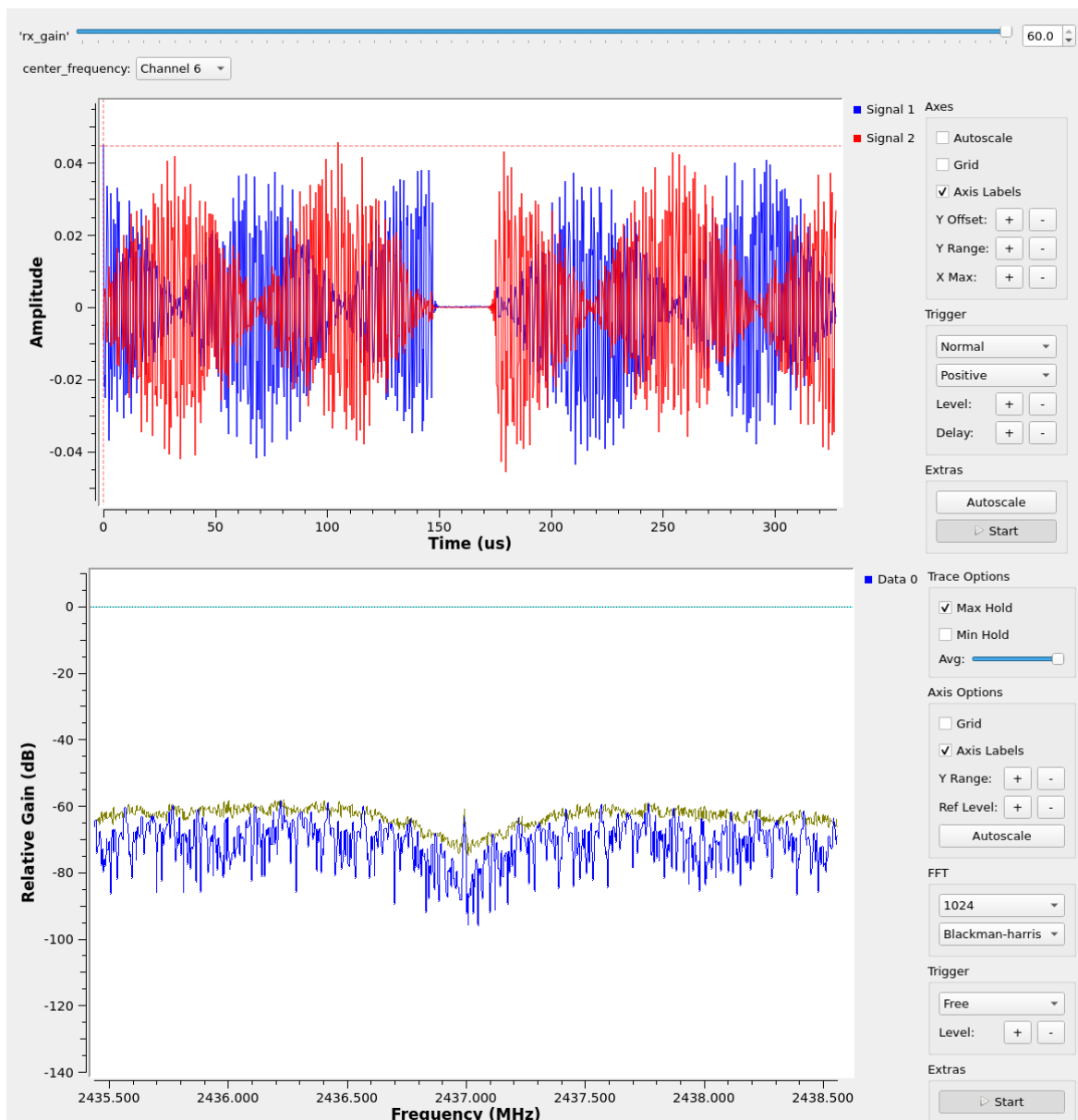
Question 5

For the measurements, we used `addr = 10.61.40.141` (Group 3), Channel 6, and a value of `rx_gain = 30dB`.

Using the trigger controls, we found the peak time-domain magnitude to be 0.002 during a burst. Using the max hold in the Frequency Sink, we found that the max spectral value is at -85.63dB. We also visually found the approximate spectral baseline to be around -100dB. Time was displayed in microseconds, relative gain was displayed in dB, and frequency was displayed in MHz while amplitude was unitless.



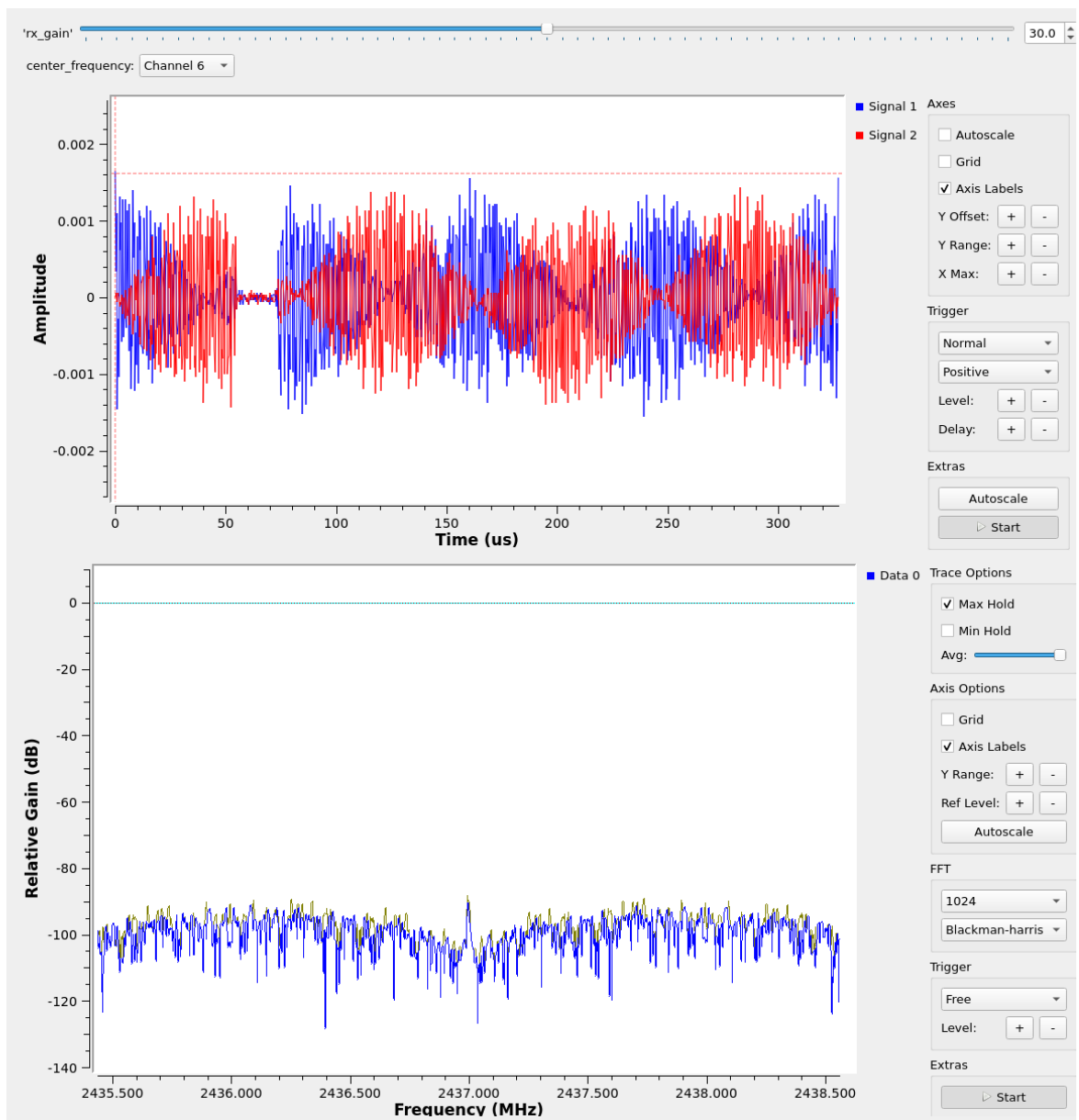
Increasing the RX gain resulted in a higher received signal magnitude in the time domain. The spectral baseline and the maximum spectral value also increased, indicating that both the signal and noise components were being amplified by the receiver. The plots below illustrate the system response when the RX gain was increased to 60 dB. The signal magnitude went up substantially to 0.04, the maximum spectral power to around -60 dB, and the approximate spectral baseline was around -72 dB.

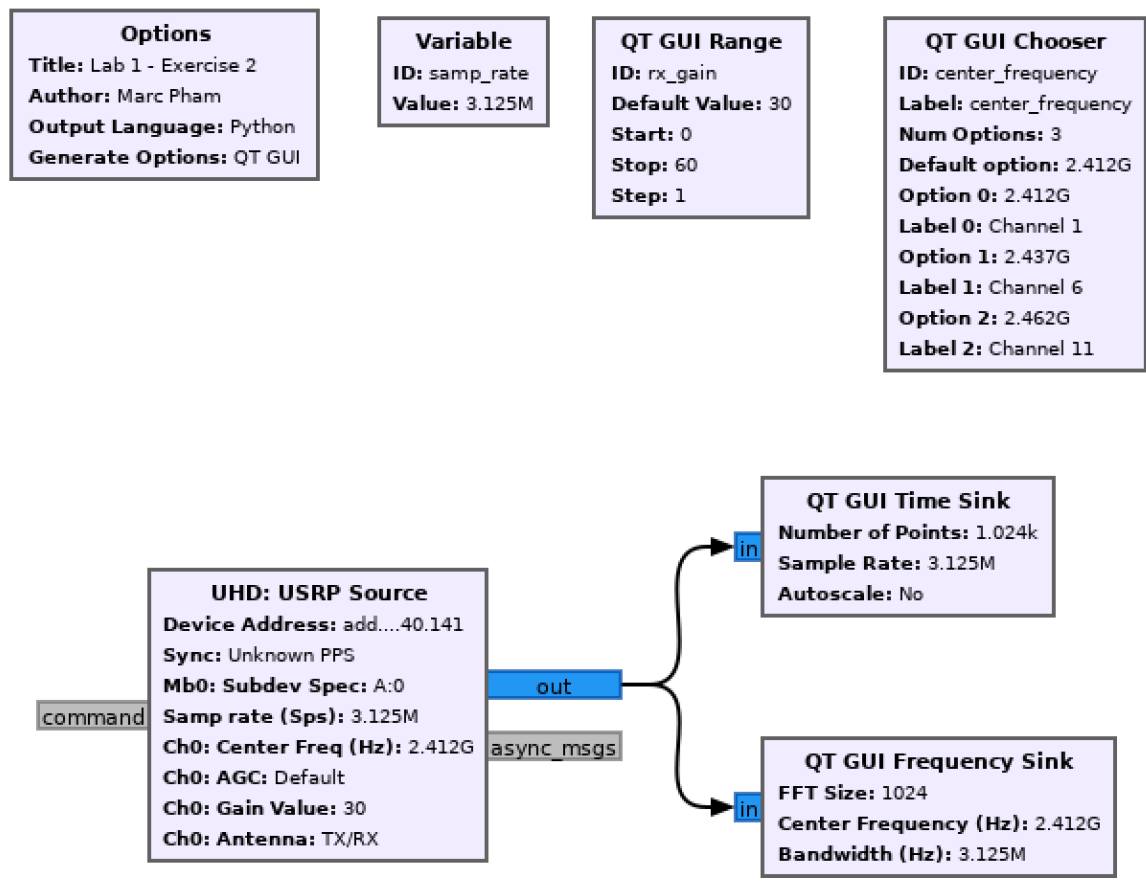


Question 6

We controlled the following variables: the channel to Channel 6, rx_gain to 30 dB, and the sample rate to 3.125 MS/s as instructed. Using Group 4's RX IP 10.61.40.174, we found the peak time-domain magnitude to be 0.0017, approximate spectral baseline to be -102 dB, and the max spectral value to be -87.03 dB. Overall, we observed small but minimal differences in all three values. There was a 0.0003 difference in peak time-domain magnitude, 2dB difference in approximate spectral baseline, and 1.40 dB difference in max spectral value

These differences may be caused by several factors. Antenna response can vary slightly between antennas, causing one antenna to receive more signal power than another. RF-chain response differences can occur because the receiver components, such as amplifiers and filters, are not perfectly identical and may introduce different amounts of gain or noise. Traffic variation can affect the results because activity changes continuously as devices transmit data, causing signal levels to fluctuate over time. Multipath propagation occurs when radio waves reflect off walls and objects before reaching the receiver. These reflected signals can combine with the direct signal, either strengthening or weakening the received signal and resulting in different measurements.



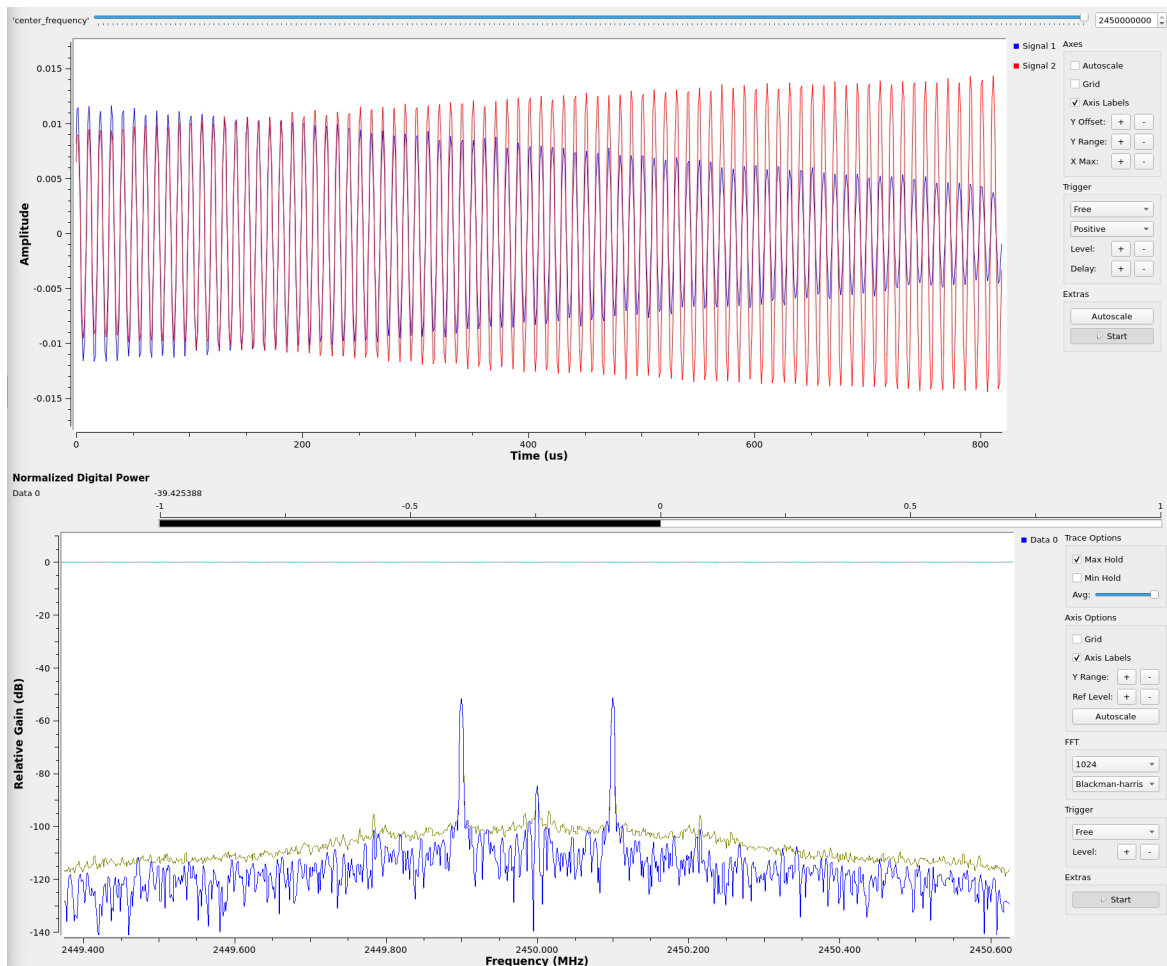


Exercise 3: Transmitting and Receiving a Sine Wave

5.1 One Tone and Transmit Gain

Question 4

For Question 4 and 5, we used the Group 4 transmitter and receiver.



The receiver spectrum contains three prominent peaks. The two largest peaks appear at approximately 2.4499 GHz and 2.4501 GHz, which are the expected frequencies produced when a 100 kHz baseband tone is transmitted around a 2.45 GHz carrier frequency.

To calculate the two expected RF frequencies, we can show that the original sine wave signal $x(t)$ has two components with those expected RF frequencies.

$x(t) = 0.1 \sin(2\pi f_{bt}) = \frac{0.1}{2j} (e^{j2\pi f_{bt}} - e^{-j2\pi f_{bt}})$ using the trig identity representing sine as Euler's identity $\sin(\theta) = \frac{e^{j\theta} - e^{-j\theta}}{2j}$

USRP shifts the spectrum of the baseband signal from 0 Hz to the center frequency $f_c = 2.45$ GHz, so the produced signal $x_{\text{USRP}}(t)$ can be effectively transmitted over the air via antenna.

$x_{\text{USRP}}(t) = \frac{0.1}{2j} (e^{j2\pi f_{bt}} - e^{-j2\pi f_{bt}}) e^{j2\pi f_{ct}} = \frac{0.1}{2j} (e^{j2\pi (f_c + f_b)t} - e^{j2\pi (f_c - f_b)t})$

Thus, the produced signal has two signal components with expected frequencies $f_c + f_b = 2.45 \text{ GHz} + 100 \text{ kHz} = 2.4501 \text{ GHz}$ and $f_c - f_b = 2.45 \text{ GHz} - 100 \text{ kHz} = 2.4499 \text{ GHz}$

A third somewhat prominent peak appears at 2.450 GHz, which corresponds to the center-frequency leakage. The USRP uses a 2.45 GHz carrier signal to shift the baseband spectrum from around 0 Hz to around 2.45 GHz so it can be transmitted over the air. Because the transmitter hardware is not perfect, a small amount of the carrier signal itself leaks into the transmitted waveform, producing a peak at exactly 2.45 GHz in the received spectrum.

Several much smaller peaks (spurs) appear around the main tones due to non-ideal hardware components in the transmitter and receiver, which introduce unwanted frequency components at nearby frequencies.

Question 5

When the TX gain varied between 40 dB, 45 dB, and 50 dB, the measured received normalized power was -39.425388 dB, -34.016899 dB, and -29.057222 dB respectively. For an increase in 1 dB in TX gain, there was approximately a 1.037 dB increase in received normalized power.

```
In [86]: tx_gain = np.array([40, 45, 50])
power_received = np.array([-39.425388, -34.016899, -29.057222])
```

```
In [89]: # Linear fit: y = mx + b
m, b = np.polyfit(tx_gain, power_received, 1)

# Generate fitted line
x_fit = np.linspace(tx_gain.min(), tx_gain.max(), 100)
y_fit = m * x_fit + b

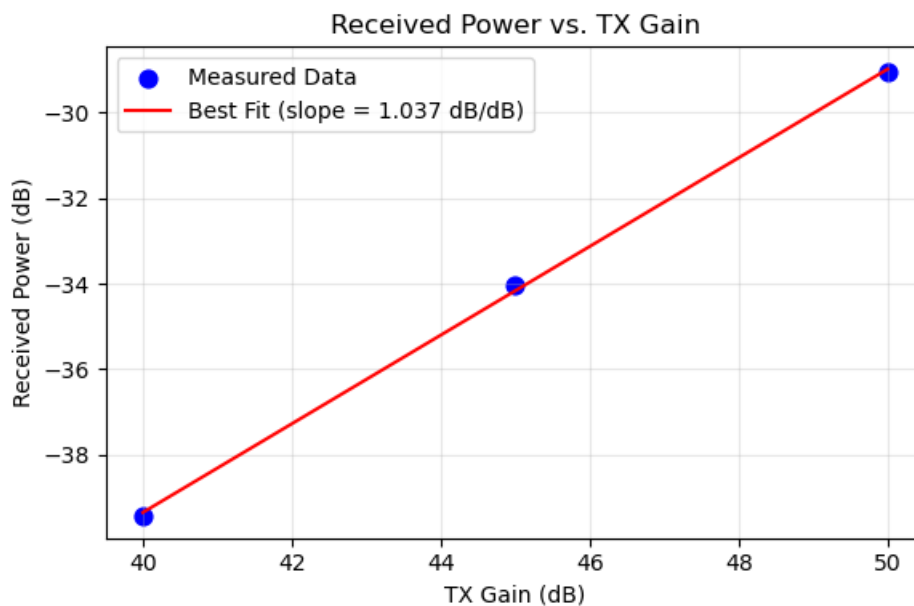
plt.figure(figsize=(6, 4))

# Data points
plt.scatter(tx_gain, power_received,
            color='blue', s=60, label='Measured Data')

# Best-fit line
plt.plot(x_fit, y_fit,
         color='red',
         label=f'Best Fit (slope = {m:.3f} dB/dB)')

plt.xlabel('TX Gain (dB)')
plt.ylabel('Received Power (dB)')
plt.title('Received Power vs. TX Gain')
plt.grid(True, alpha=0.3)
plt.legend()

plt.tight_layout()
plt.show()
```



While the transmitter, receiver, and propagation path remain in their linear operating regions, the received power is expected to increase by approximately the same amount as the TX gain. Therefore, a 5 dB increase in TX gain should produce about a 5 dB increase in received power.

For the 5 dB TX-gain step from 40 dB to 45 dB, the received power increased by 5.408489 dB.

```
In [91]: np.round(power_received[1] - power_received[0], 6)
```

```
Out[91]: 5.408489
```

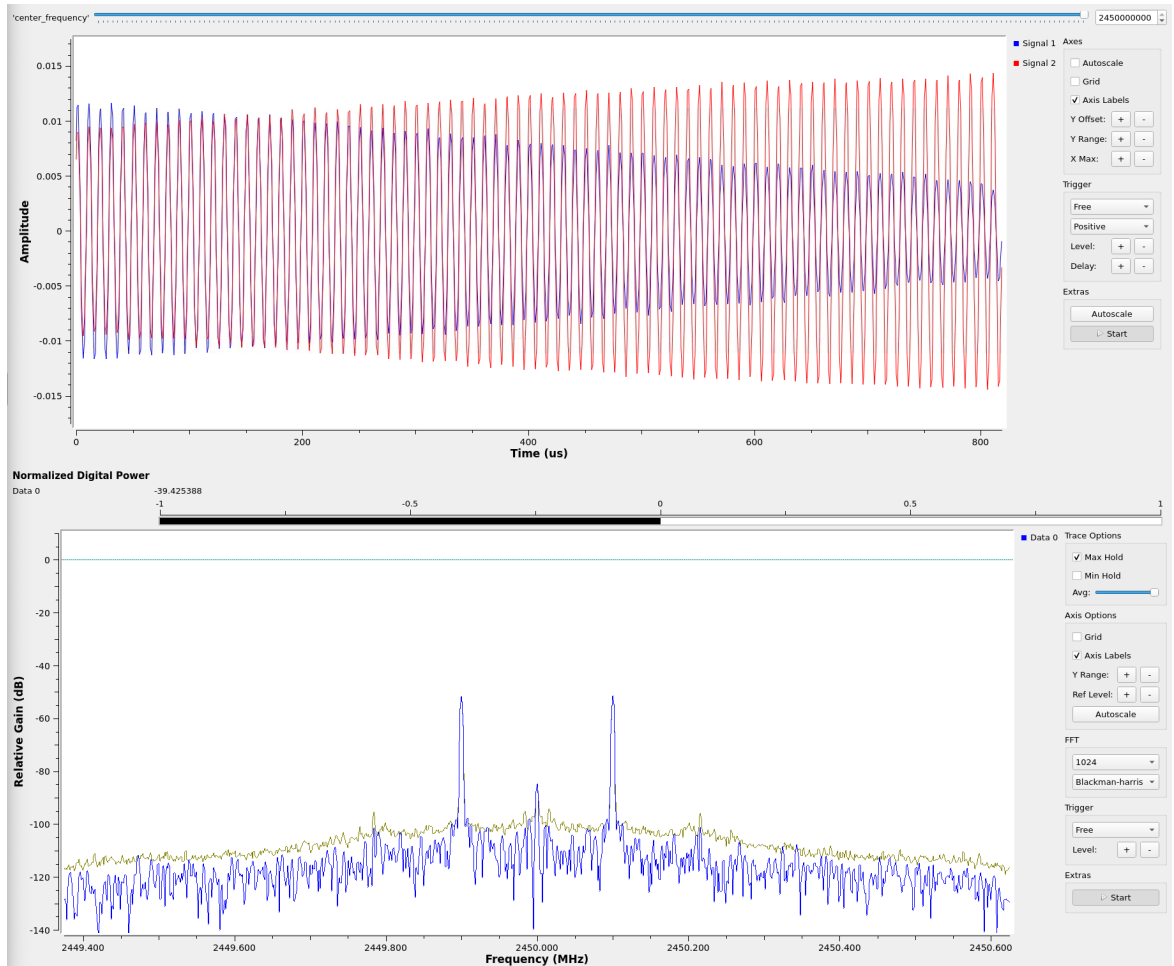
For the 5 dB TX-gain step from 45 dB to 50 dB, the received power increased by 4.959677 dB.

```
In [92]: np.round(power_received[2] - power_received[1], 6)
```

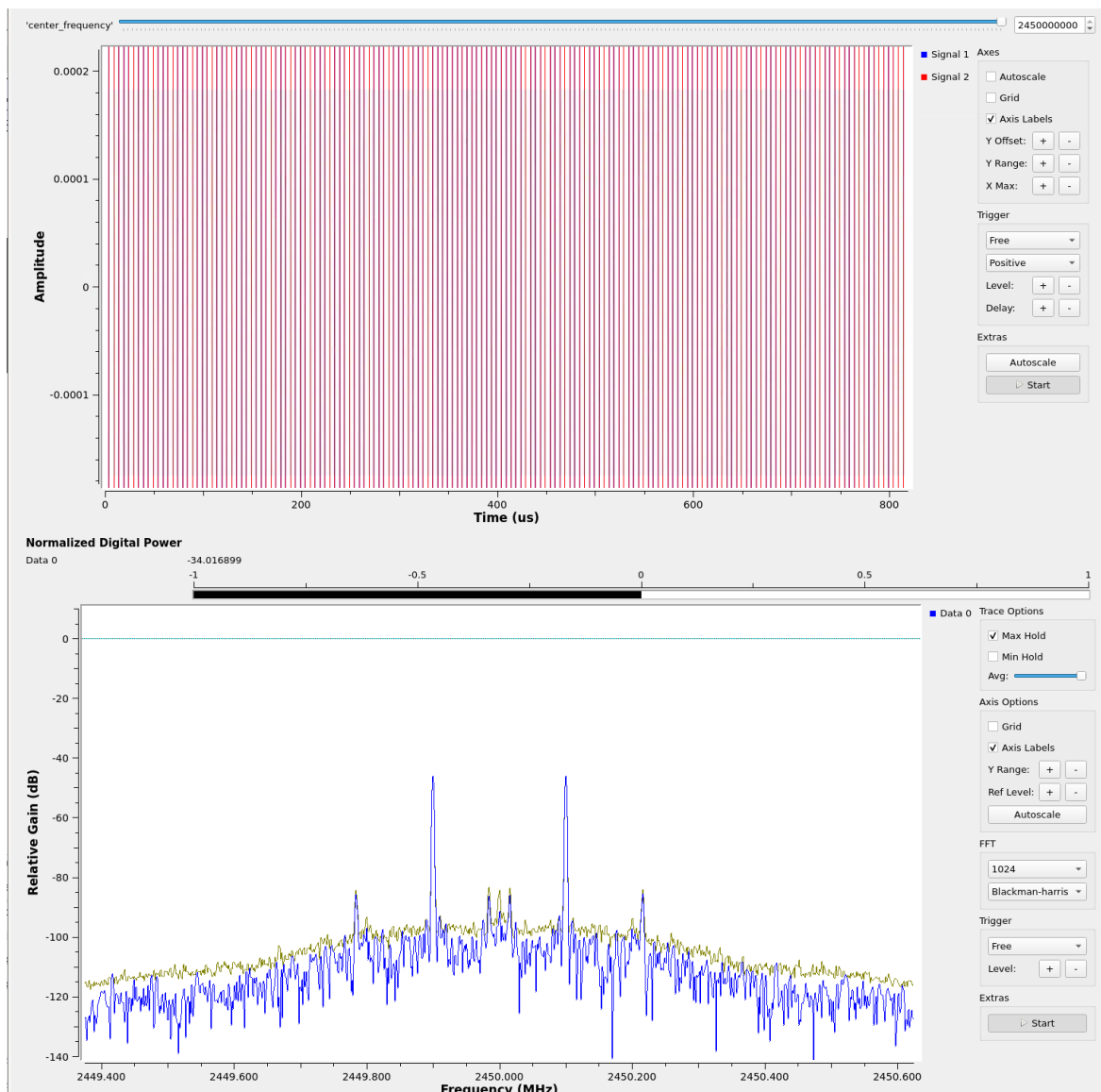
Out[92]: 4.959677

The change in normalized power received (5.408489 dB and 4.959677 dB) closely matches the expected 5 dB increase, supporting an approximately linear 1:1 relationship between the TX gain and received normalized power. This is because of the link budget relationship, where increasing the transmitted power by a given number of decibels should add the same number of decibels to the received signal level if the channel conditions are kept constant.

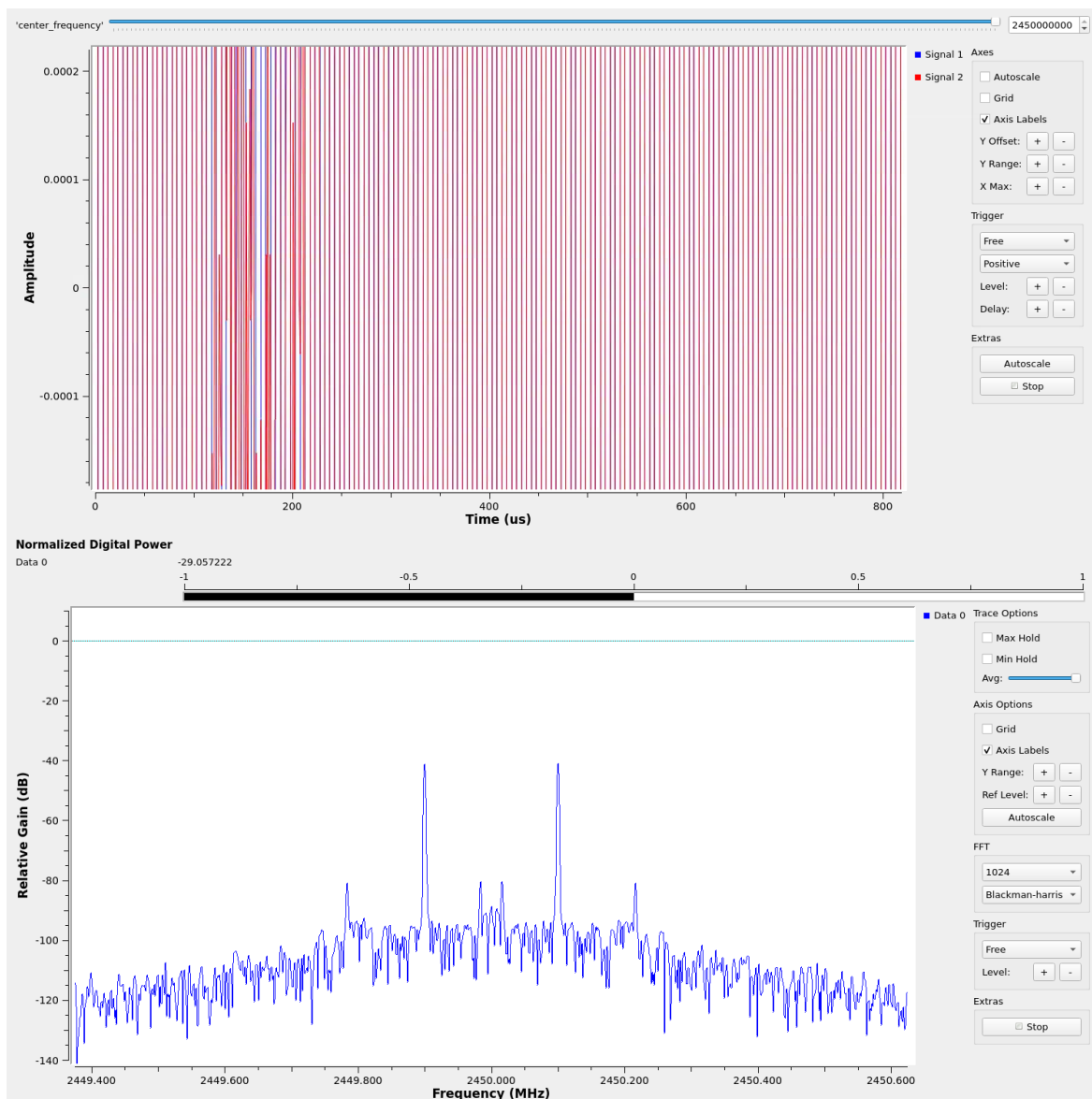
Graph for a Gain of 40 dB



Graph for a Gain of 45 dB



Graph for a Gain of 50 dB

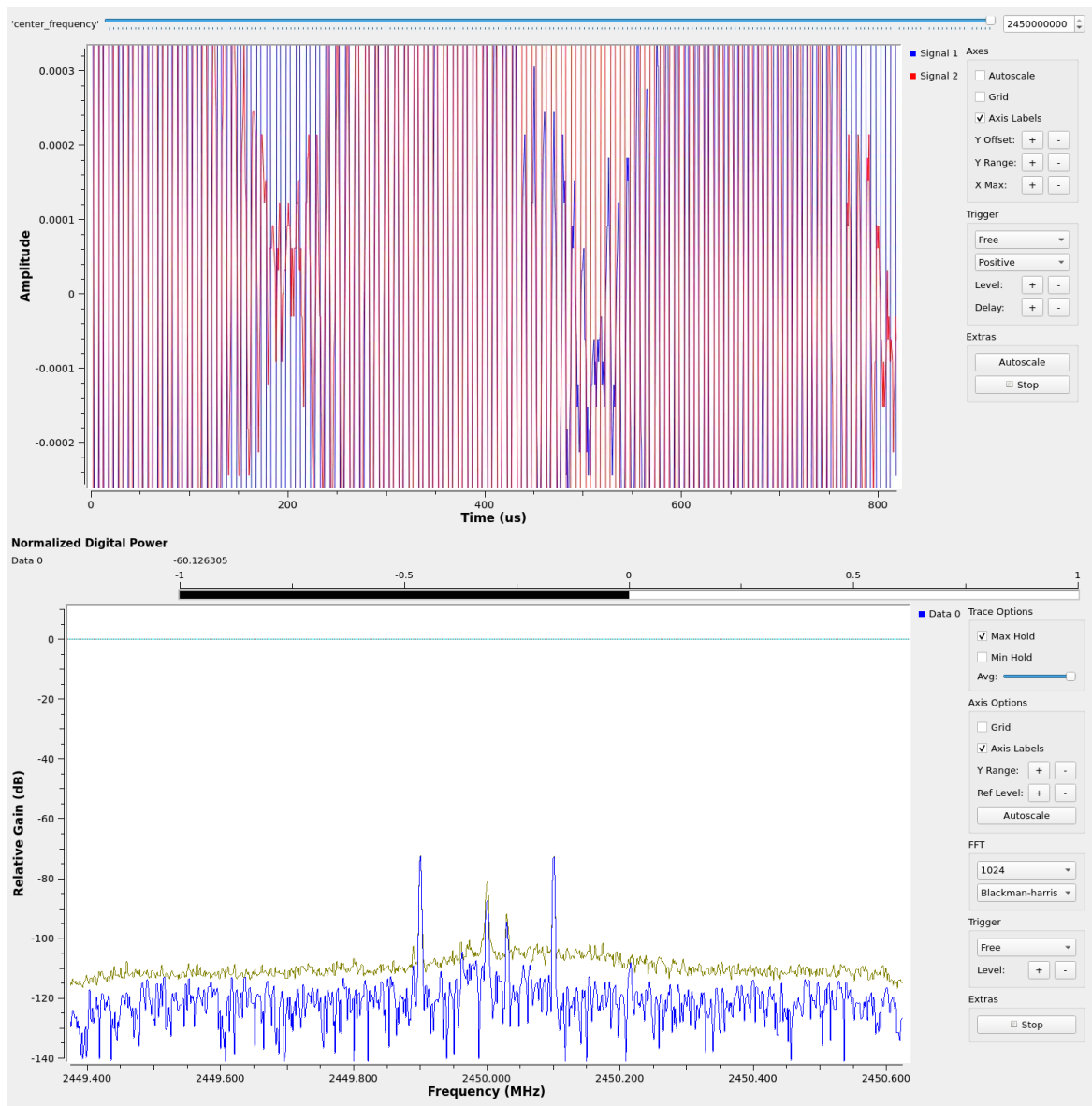


Question 6

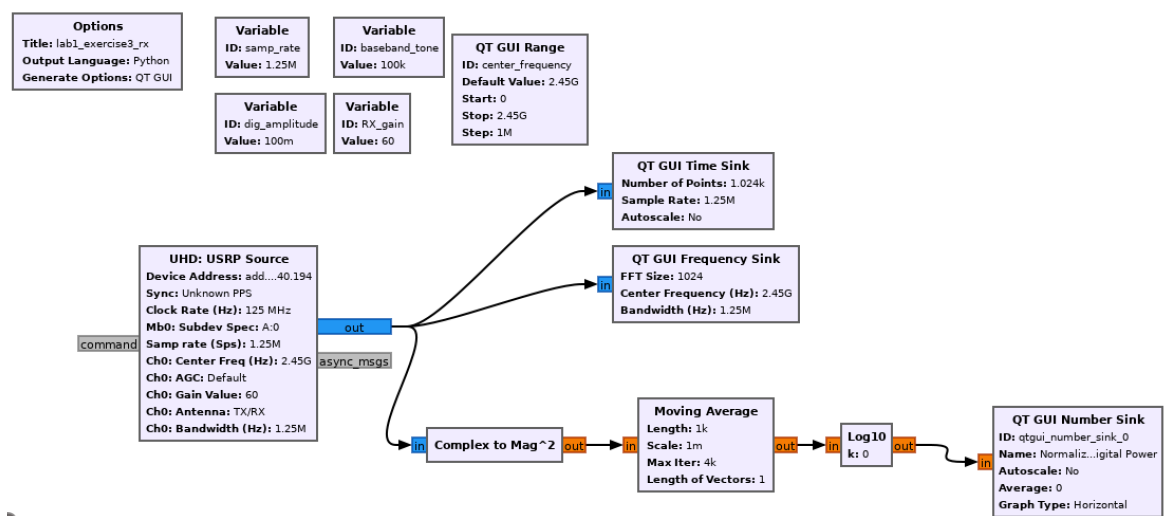
For our second transmitter-receiver pair, we used the Group 5 receiver and transmitter to repeat one gain point where TX gain is 40 dB. We measured the normalized digital power to be -60.126305 dB, which differs from the -39.425388 dB measured using the Group 4 receiver and transmitter with TX gain = 40 dB. This corresponds to a decrease of 20.700917 dB from the Group 4 receiver and transmitter to the Group 5 receiver and transmitter. This difference is likely due to variations in antenna placement and orientation and hardware-to-hardware differences.

```
In [94]: np.round(-39.425388+60.126305, 6)
```

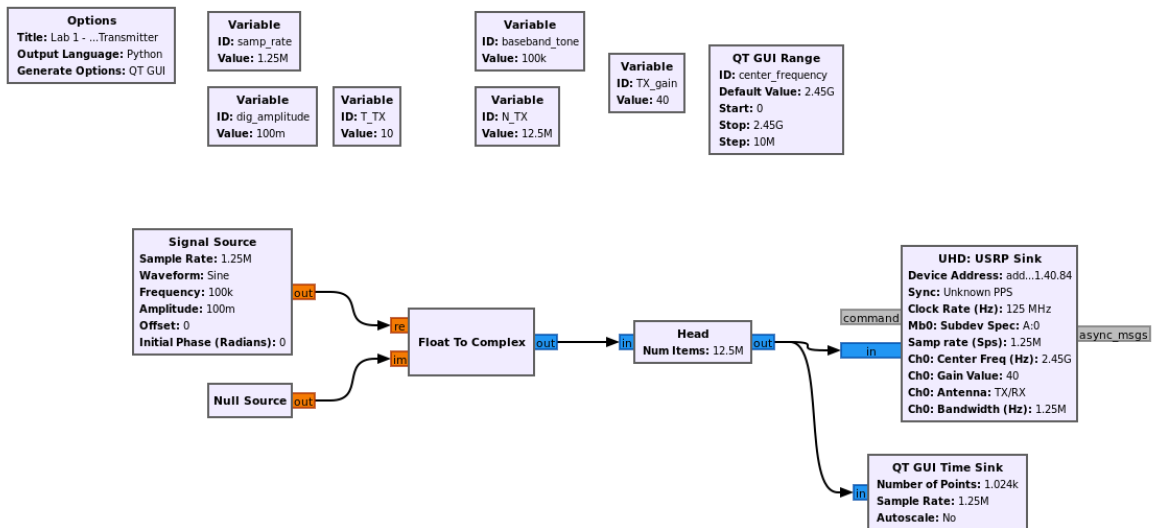
```
Out[94]: 20.700917
```



Receiver Flowgraph



Transmitter Flowgraph



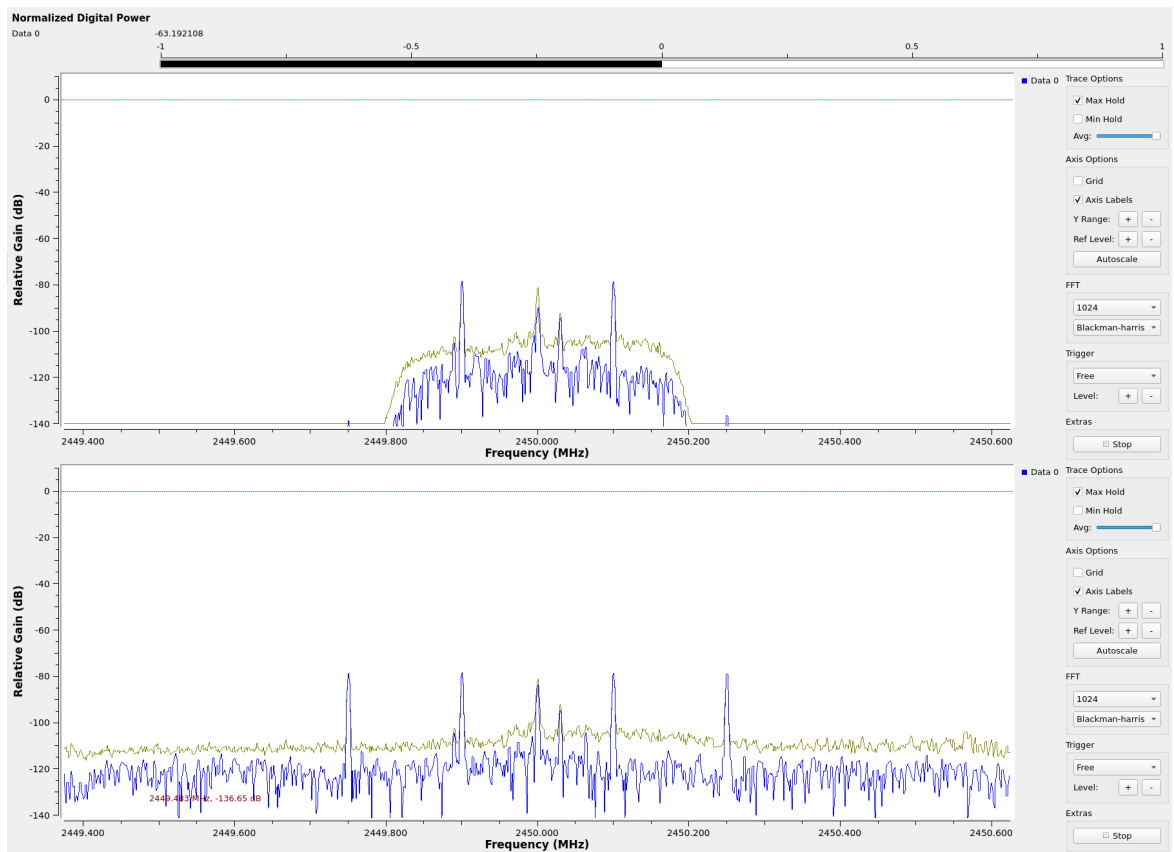
5.2 Two Tones and Filtering

The Group 5 receiver and transmitter were used for exercise 5.2

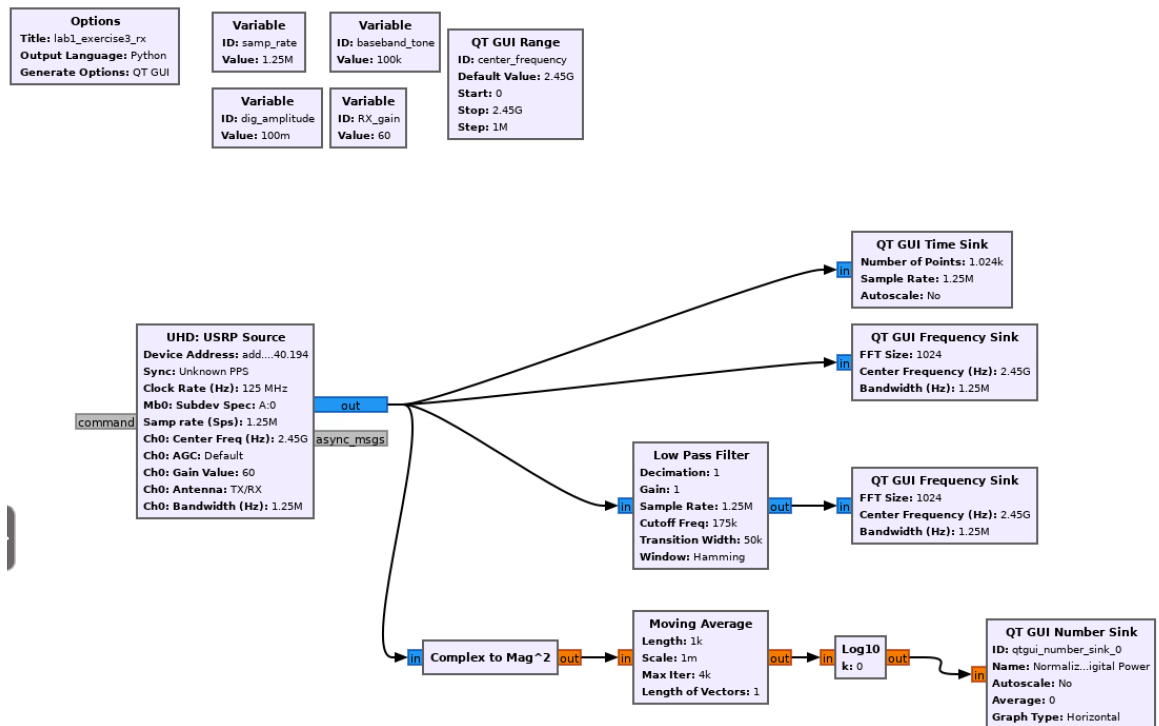
Question 3

The unfiltered spectra has four main peaks at frequencies 2449.75 and 2450.25 MHz for the 250 kHz baseband tone and 2449.90 and 2450.10 MHz for the 100kHz baseband tone. Since each real-valued tone produces upper and lower sidebands about the 2.45 GHz carrier frequency, two peaks are observed for each transmitted tone.

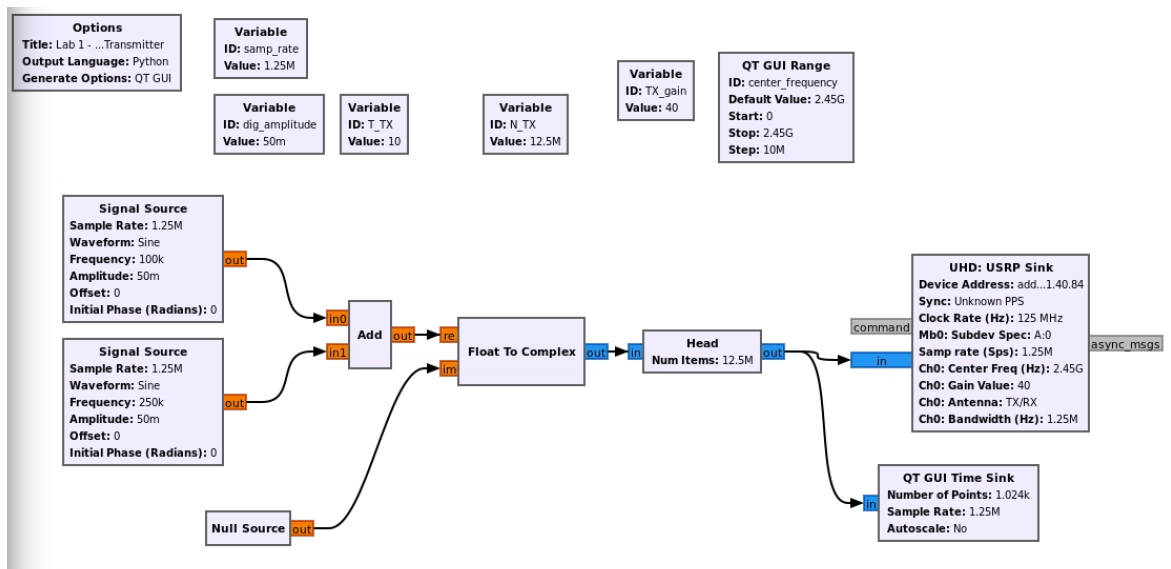
The filtered spectra, compared to the unfiltered spectra, removes the peaks associated with the 250 kHz baseband tone while retaining the peaks associated with the 100 kHz tone. This occurs because the low-pass filter has a cutoff frequency of 175 kHz, which places the 100 kHz tone within the filter's passband and the 250 kHz tone within its stopband. The 50 kHz transition width provides a gradual transition between the passband and stopband rather than an abrupt cutoff. As a result, the 100 kHz tone is preserved while the 250 kHz tone is strongly attenuated.



Receiver Flowgraph



Transmitter Flowgraph



Short Discussion of Issues

We ran into a couple of issues getting started. The antennas had not yet been attached to the SDRs, so we had to wait for that to be completed before collecting data. There was also a Qt library conflict on the server that caused problems with the QT GUI sinks in GNU Radio. After these issues were resolved, the receiver was able to display results normally and the rest of the lab proceeded without any major problems.